

Reinforcement Learning

n -step Bootstrapping

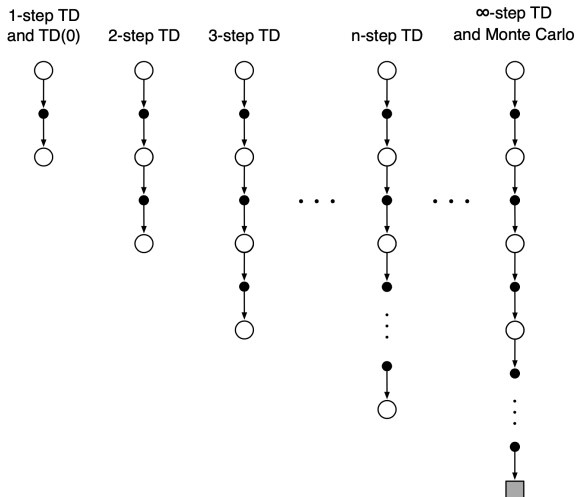
Min-hwan Oh

Seoul National University

n -step TD Prediction

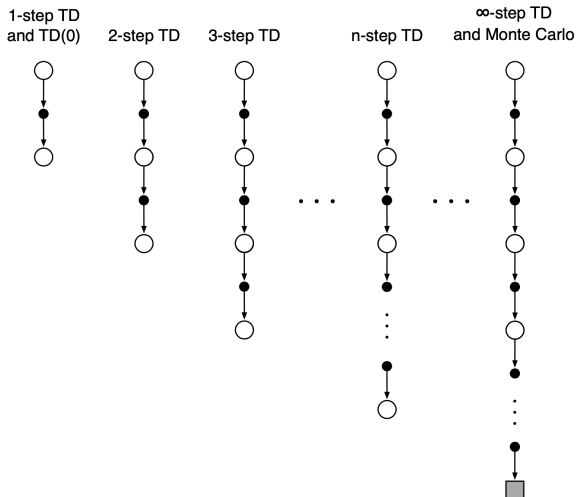
- Monte Carlo (MC) methods perform an update for each visited state based on the entire sequence of rewards from that state until the end of the episode.
- One-step TD methods update is based on the next reward only.
- n -step: in-between MC and one-step TD

One-step TD vs. n -step TD vs. MC



- Why is n -step TD still TD, not MC?

One-step TD vs. n -step TD vs. MC



- Why is n -step TD still TD, not MC? Since we bootstrap! i.e., we change an earlier estimate based on how it differs from a later estimate (only that the later estimate is now n step later)

Targets for TD(0) and MC

Notation $G_t^{(n)} = G_{t:t+n}$ the return from time step t to $t + n$

$$n = 1 \quad (TD(0)) \qquad G_t^{(1)} = R_{t+1} + \gamma V_t(S_{t+1})$$

$$n = 2 \qquad G_t^{(2)} = R_{t+1} + \gamma R_{t+2} + \gamma^2 V_{t+1}(S_{t+2})$$

$$\vdots \qquad \qquad \qquad \vdots$$

$$n = \infty \quad (MC) \qquad G_t^{(\infty)} = R_{t+1} + \gamma R_{t+2} + \dots + \gamma^{T-1} R_T$$

- MC target is a sampled return
- one-step TD target is a one-step lookahead value + immediate reward

n -step TD Target

$$G_t^{(n)} = R_{t+1} + \gamma R_{t+2} + \dots + \gamma^{n-1} R_{t+n} + \gamma^n V_{t+n-1}(S_{t+n})$$

- All n -step returns can be considered approximations of the full return, truncated after n steps and then corrected for the remaining steps by $V_{t+n-1}(S_{t+n})$.
- n -steps return for $n > 1$ involve future rewards that are not available from the transition $t \rightarrow t + 1$. We'll see eligibility traces (chapter 12) for an online method not involving future rewards.
- Plugging this into the state-value learning:

$$V_{t+n}(S_t) := V_{t+n-1}(S_t) + \alpha[G_t^{(n)} - V_{t+n-1}(S_t)]$$

- No changes are made during the first $n - 1$ steps of each episodes. To make up for that, an equal number of updates are made after the termination of the episode, before starting the next.

n -step TD Prediction Pseudocode

n -step TD for estimating $V \approx v_\pi$

Input: a policy π

Algorithm parameters: step size $\alpha \in (0, 1]$, a positive integer n

Initialize $V(s)$ arbitrarily, for all $s \in \mathcal{S}$

All store and access operations (for S_t and R_t) can take their index mod $n + 1$

Loop for each episode:

 Initialize and store $S_0 \neq$ terminal

$T \leftarrow \infty$

 Loop for $t = 0, 1, 2, \dots$:

 If $t < T$, then:

 Take an action according to $\pi(\cdot | S_t)$

 Observe and store the next reward as R_{t+1} and the next state as S_{t+1}

 If S_{t+1} is terminal, then $T \leftarrow t + 1$

$\tau \leftarrow t - n + 1$ (τ is the time whose state's estimate is being updated)

 If $\tau \geq 0$:

$G \leftarrow \sum_{i=\tau+1}^{\min(\tau+n, T)} \gamma^{i-\tau-1} R_i$

 If $\tau + n < T$, then: $G \leftarrow G + \gamma^n V(S_{\tau+n})$ ($G_{\tau:\tau+n}$)

$V(S_\tau) \leftarrow V(S_\tau) + \alpha [G - V(S_\tau)]$

 Until $\tau = T - 1$

Error-Reduction Property

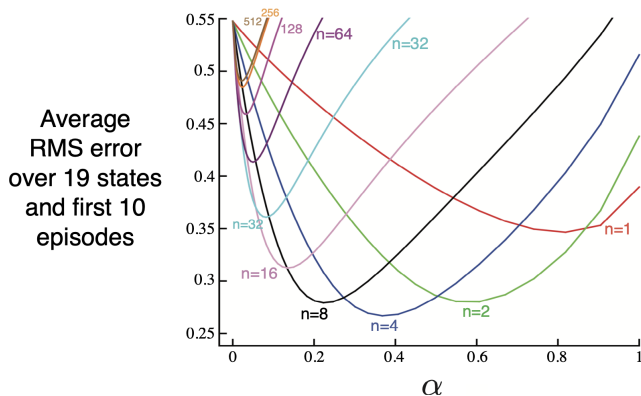
- The n -step return uses the value function V_{t+n-1} to correct for the missing rewards beyond R_{t+n} .
- The expectation of n -step returns is guaranteed to be a better estimate of v_π than V_{t+n-1} (in a worst-state sense)

$$\max_s |\mathbb{E}_\pi[G_t^{(n)} | S_t = s] - v_\pi(s)| \leq \gamma^n \max_s |V_{t+n-1}(s) - v_\pi(s)|$$

- This is called the **error reduction property**.
- We can use this to show that all n -steps methods converge to the correct predictions under appropriate technical conditions.

n -step TD Performances

- Effect of n in the performances of n -step TD.



n -step Sarsa

- Use n -step not just for prediction but also for control
- Switch states for state-action pairs and use a ϵ -greedy policy
- The backup diagrams for n -step Sarsa are like those of n -step TD except that the Sarsa ones start and end in a state-action node
- We redefine the n -step return in terms of estimated action-values instead of state-values:

$$G_{t:t+n} := R_{t+1} + \gamma R_{t+2} + \dots + \gamma^{n-1} R_{t+n} + \gamma^n Q_{t+n-1}(S_{t+n}, A_{t+n})$$

- And plug that into our GPI update:

$$Q_{t+n}(S_t, A_t) \leftarrow Q_{t+n-1}(S_t, A_t) + \alpha(G_{t:t+n} - Q_{t+n-1}(S_t, A_t))$$

- The values of all other states remain unchanged,
 $Q_{t+n}(s, a) = Q_{t+n-1}(s, a)$ for all $(s, a) \neq (S_t, A_t)$

n -step Sarsa Pseudocode

n -step Sarsa for estimating $Q \approx q_*$ or q_π

Initialize $Q(s, a)$ arbitrarily, for all $s \in \mathcal{S}, a \in \mathcal{A}$

Initialize π to be ε -greedy with respect to Q , or to a fixed given policy

Algorithm parameters: step size $\alpha \in (0, 1]$, small $\varepsilon > 0$, a positive integer n

All store and access operations (for S_t , A_t , and R_t) can take their index mod $n + 1$

Loop for each episode:

 Initialize and store $S_0 \neq \text{terminal}$

 Select and store an action $A_0 \sim \pi(\cdot | S_0)$

$T \leftarrow \infty$

 Loop for $t = 0, 1, 2, \dots$:

 If $t < T$, then:

 Take action A_t

 Observe and store the next reward as R_{t+1} and the next state as S_{t+1}

 If S_{t+1} is terminal, then:

$T \leftarrow t + 1$

 else:

 Select and store an action $A_{t+1} \sim \pi(\cdot | S_{t+1})$

$\tau \leftarrow t - n + 1$ (τ is the time whose estimate is being updated)

 If $\tau \geq 0$:

$G \leftarrow \sum_{i=\tau+1}^{\min(\tau+n, T)} \gamma^{i-\tau-1} R_i$

 If $\tau + n < T$, then $G \leftarrow G + \gamma^n Q(S_{\tau+n}, A_{\tau+n})$ ($G_{\tau:\tau+n}$)

$Q(S_\tau, A_\tau) \leftarrow Q(S_\tau, A_\tau) + \alpha [G - Q(S_\tau, A_\tau)]$

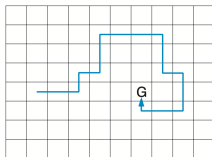
 If π is being learned, then ensure that $\pi(\cdot | S_\tau)$ is ε -greedy wrt Q

 Until $\tau = T - 1$

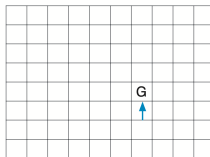
n -step Sarsa in Gridworld

- Gridworld example of the speedup of policy learning due to the use of n -step methods
- The values were all initially 0, and all rewards were zero except for a positive reward at **G**

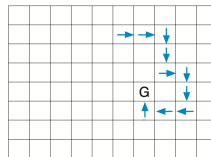
Path taken



Action values increased
by one-step Sarsa



Action values increased
by 10-step Sarsa



n -step Sarsa Backup Diagram

1-step Sarsa
aka Sarsa(0)



2-step Sarsa



3-step Sarsa



n -step Sarsa

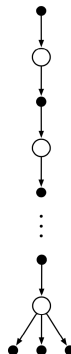


...

∞ -step Sarsa
aka Monte Carlo



n -step
Expected Sarsa



n -step Expected Sarsa

- What about n -step version of expected Sarsa?
- The backup diagram consists of a linear string of sample actions and states and the last element is a branch over all action possibilities, weighted by their probability of occurring under π . The n -step return is here defined by:

$$G_{t:t+n} = R_{t+1} + \gamma R_{t+2} + \dots + \gamma^{n-1} R_{t+n} + \gamma^n \bar{V}_{t+n-1}(S_{t+n})$$

where $\bar{V}_t(s)$ is the expected approximate value of state s , using the estimated action values and the policy:

$$\bar{V}_t(s) := \sum_a \pi(a|s) Q_t(s, a)$$

- Expected approximate values are important!

n -step Off Policy Learning by Importance Sampling

- In order to use the data from the behaviour policy b , we must take into account the relative probability of taking the actions that were taken.
- In n -step methods the returns are constructed over n steps so we are interested in the relative probability of just these n actions.
- For example to make a simple version of n -step TD, we can weight the update for time t (made at time $t + n$) by the importance-sampling ratio:

$$\rho_{t:h} = \prod_{k=t}^{\min(h, T-1)} \frac{\pi(A_k \| S_k)}{b(A_k \| S_k)}$$

- We plug this into our update we get:

$$V_{t+n}(S_t) \leftarrow V_{t+n-1}(S_t) + \alpha \rho_{t:t+n-1} (G_{t:t+n} - V_{t+n-1}(S_t))$$

- n -step Sarsa update can be extended for the off-policy method:

$$Q_{t+n}(S_t, A_t) \leftarrow Q_{t+n-1}(S_t, A_t) + \alpha \rho_{t+1:t+n-1} (G_{t:t+n} - Q_{t+n-1}(S_t, A_t))$$

Off-Policy n -step Sarsa Pseudocode

Off-policy n -step Sarsa for estimating $Q \approx q_*$ or q_π

Input: an arbitrary behavior policy b such that $b(a|s) > 0$, for all $s \in \mathcal{S}, a \in \mathcal{A}$

Initialize $Q(s, a)$ arbitrarily, for all $s \in \mathcal{S}, a \in \mathcal{A}$

Initialize π to be greedy with respect to Q , or as a fixed given policy

Algorithm parameters: step size $\alpha \in (0, 1]$, a positive integer n

All store and access operations (for S_t , A_t , and R_t) can take their index mod $n + 1$

Loop for each episode:

 Initialize and store $S_0 \neq \text{terminal}$

 Select and store an action $A_0 \sim b(\cdot|S_0)$

$T \leftarrow \infty$

 Loop for $t = 0, 1, 2, \dots$:

 If $t < T$, then:

 Take action A_t

 Observe and store the next reward as R_{t+1} and the next state as S_{t+1}

 If S_{t+1} is terminal, then:

$T \leftarrow t + 1$

 else:

 Select and store an action $A_{t+1} \sim b(\cdot|S_{t+1})$

$\tau \leftarrow t - n + 1$ (τ is the time whose estimate is being updated)

 If $\tau \geq 0$:

$\rho \leftarrow \prod_{i=\tau+1}^{\min(\tau+n, T-1)} \frac{\pi(A_i|S_i)}{b(A_i|S_i)}$ ($\rho_{\tau+1:\tau+n}$)

$G \leftarrow \sum_{i=\tau+1}^{\min(\tau+n, T)} \gamma^{i-\tau-1} R_i$

 If $\tau + n < T$, then: $G \leftarrow G + \gamma^n Q(S_{\tau+n}, A_{\tau+n})$ ($G_{\tau:\tau+n}$)

$Q(S_\tau, A_\tau) \leftarrow Q(S_\tau, A_\tau) + \alpha \rho [G - Q(S_\tau, A_\tau)]$

 If π is being learned, then ensure that $\pi(\cdot|S_\tau)$ is greedy wrt Q

 Until $\tau = T - 1$

Off-Policy without Importance Sampling?

- Q-learning is an off-policy algorithm but does not use importance sampling! How?

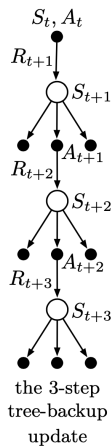
Off-Policy without Importance Sampling?

- Q-learning is an off-policy algorithm but does not use importance sampling! How?
- Recall Q-learning update rule:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t) \right]$$

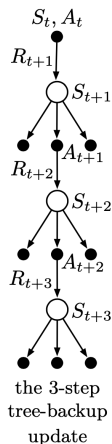
- This update rule actually matches exactly what the “target policy” (greedy policy in this case) does.
- The target is not drawn from a behavior policy. Consequently, we don’t need to correct for different policy distributions!

Off-Policy without Importance Sampling



- Off-policy without importance sampling: Q-learning an Expected Sarsa do this for the one-step TD, but for n -step TD?

Off-Policy without Importance Sampling



- Off-policy without importance sampling: Q-learning an Expected Sarsa do this for the one-step TD, but for n -step TD?
- So far the target for the update of a value function was combining the rewards along the way and the estimated values of the nodes at the bottom.
- We add to this the estimated values of the actions not selected
- Update from the entire tree of estimated action values.

The n -step Backup Tree Algorithm

- Each leaf node contributes to the target with a weight proportional to their probability of occurring under π .
- Each first-level action a contributes with a weight of $\pi(a|S_{t+1})$
- The action actually taken, A_{t+1} does not contribute and its probability $\pi(A_{t+1}|S_{t+1})$ is used to weight all the second level action values.
- Each second-level action a' has a weight of $\pi(A_{t+1}|S_{t+1})\pi(a'|S_{t+2})$.
- The one-step return (target) is (for $t < T - 1$)

$$G_{t:t+1} := R_{t+1} + \gamma \sum_a \pi(a|S_{t+1}) Q_t(S_{t+1}, a)$$

- And the two-step tree-backup return is (for $t < T - 2$)

$$\begin{aligned} G_{t:t+2} &:= R_{t+1} + \gamma \sum_{a \neq A_{t+1}} \pi(a|S_{t+1}) Q_{t+1}(S_{t+1}, a) \\ &\quad + \gamma \pi(A_{t+1}|S_{t+1}) \left(R_{t+2} + \gamma \sum_a \pi(a|S_{t+2}) Q_{t+1}(S_{t+2}, a) \right) \\ &= R_{t+1} + \gamma \sum_{a \neq A_{t+1}} \pi(a|S_{t+1}) Q_{t+1}(S_{t+1}, a) + \gamma \pi(A_{t+1}|S_{t+1}) G_{t+1:t+2} \end{aligned}$$

The n -step Backup Tree Algorithm (cont'd)

- Two-step tree backup return (for $t < T - 2$)

$$G_{t:t+2} := R_{t+1} + \gamma \sum_{a \neq A_{t+1}} \pi(a|S_{t+1}) Q_{t+1}(S_{t+1}, a) + \gamma \pi(A_{t+1}|S_{t+1}) G_{t+1:t+2}$$

- Hence, this suggests the recursive form of the n -step tree backup return (for $t < T - 1$)

$$G_{t:t+n} := R_{t+1} + \gamma \sum_{a \neq A_{t+1}} \pi(a|S_{t+1}) Q_{t+n-1}(S_{t+1}, a) + \gamma \pi(A_{t+1}|S_{t+1}) G_{t+1:t+n}$$

- We then use this target in the usual action-value update from n -step Sarsa:

$$Q_{t+n}(S_t, A_t) \leftarrow Q_{t+n-1}(S_t, A_t) + \alpha (G_{t:t+n} - Q_{t+n-1}(S_t, A_t))$$

Next class: Planning and Learning with Tabular Methods
(Chapter 8 of Sutton & Barto)